

Adaptive Particle Filter on TabArena: research log

Background

`nanoTabPFN` is a Prior-Fitted Network: pretrained on millions of synthetic tasks sampled from a generative prior over functions (random MLPs, GPs, or structural causal models — see `tfmplayground/external_priors/`), it does in-context learning at inference time on real tabular data.

This repo also contains a family of derived models built on top of a frozen `nanoTabPFN` backbone that add a **sequential latent filter**: instead of attending over one flat context, they split it into a **prior** block (processed all at once, like a standard support set) and an **update/stream** block (processed incrementally, like online Bayesian updating). The `AdaptiveKParticleFilter` variant runs `K` such filters in parallel as "particles" tracking different hypotheses, plus a learned **ambiguity gate** that decides how much to trust the particle ensemble vs. fall back to plain vanilla `nanoTabPFN`.

The starting question: **the TabArena evaluation script only ever used a fixed 256-row context (128 prior + 128 update), regardless of how much training data a real dataset actually had.** That can't answer "does more context help this model" — the whole point of the sequential-update mechanism. This research thread starts from fixing that and ends up investigating why the adaptive gate doesn't seem to calibrate well on real data at all.

Methods, trials, and failures

1. Fixing the TabArena eval to use all available context

`tfmplayground/experiments/evaluate_integrated_tabarena.py` hard-capped `prior_count / update_count` at 128 each (`validate_config` raised if they weren't exactly 128). Added a `--use-all-samples` flag that, per task, sizes `prior_count / update_count` to consume the *entire* training split via a new `split_prior_update_counts` helper, preserving the configured ratio.

Bug found and fixed #1: `split_prior_update_counts` 's naive rounding could put `update_count > prior_count` for odd totals (e.g. 9 rows → 4 prior / 5 update), violating an invariant the model was trained under. Fixed by flooring the update side; added a regression test sweeping totals 2–40.

Bug found and fixed #2 (the big one — a real crash): running the fixed all-samples eval crashed with the machine's 64GB RAM nearly exhausted. Root-caused via `torch.mps.driver_allocated_memory()` instrumentation (not `top`, which gave misleading readings the first time — caught by cross-checking two independent measurements): each of the four models (`vanilla`, `controlled`, `tabicl`, `adaptive`) reserves a large, **unreleased** chunk of Metal driver memory per forward call at large context (vanilla alone: **34GB** for a ~4,500-row context,

vs. **1.1GB** at the original 256-row context). Because `run()` calls all four models back-to-back for the same task without ever freeing anything, their reservations **stack**: $34 + 8.5 + 8.5 + 10 \approx 61\text{GB}$ — almost exactly the crash point. Fixed with a `release_device_memory()` helper (`torch.mps.empty_cache()`) called after every model prediction; verified directly that it collapses the peak from $\sim 61\text{GB}$ to $\sim 34\text{GB}$ (the worst single model, not the sum).

Result of the fix: full 15-dataset (of 51 TabArena tasks; the rest are regression, multiclass, or exceed size limits) all-samples run completed successfully. See Results §1.

2. Why is the adaptive gate always near-zero?

The `adaptive_k4` model's learned ambiguity was tiny everywhere (0.00003 – 0.011) and it tracked vanilla almost exactly, dataset by dataset. Tested by wiring up the model's existing `ambiguity_override` parameter and sweeping fixed alpha values ($0, 0.1, 0.25, 0.5, 0.75, 1.0$) per dataset, holding the model fixed. Found the near-zero default is *mostly* justified (8/15 datasets get strictly worse as alpha rises) but **genuinely under-confident on one dataset** (`seismic-bumps`: true optimum $\alpha \approx 0.75$, real $+0.6\%$ AUC gain the gate never captures, outputting $\alpha \approx 0.00004$ instead).

3. Does particle count (K) fix it?

Existing checkpoints trained on a *different*, richer synthetic curriculum (`train_four_mode_particle_filter.py`, $K=8$ and $K=16$, `runs/four_mode_particle_filter/`) were run through the same real 15-dataset sweep. They are dramatically more decisive (mean $\alpha \approx 0.65$ – 0.76 vs. $K=4$'s ≈ 0.00006) but only win on 6/15 datasets each — mean ROC AUC is actually *worse* than vanilla on average, because losses outweigh wins. **Confidence went up; calibration didn't.**

4. Does training scale (context size) fix it?

A quick (100-step) retrain of the $K=4$ gate at $512:512$ context instead of $128:128$. On its own internal synthetic diagnostic it almost fully passed (13/14 checks). On the real 15-dataset sweep: 5/15 wins, mean $\alpha \approx 0.117$ (much more moderate than $K=8/16$), mean ROC AUC delta ≈ -0.0006 (near-neutral — the most well-behaved of the checkpoints tested, though not the biggest winner). Did not rediscover the large `seismic-bumps` signal.

(Prior:update ratio variation was proposed as a next step but not yet run.)

5. What is the "four-mode" synthetic task, actually?

A synthetic hypothesis-tracking benchmark, **not** a stand-in for real tabular data: 1 feature, a hidden discrete "mode" (one of 4 fixed combinations of two region→label rules), a stream of clustered evidence, and conditions like `contradictory` (mode switches mid-stream — tests change detection) and `noisy` (label-flip robustness). The frozen nanoTabPFN backbone itself handles arbitrary feature counts by design (per-cell shared linear embedding, not a fixed input width) — but the **gate/particle mechanism** trained on top of it was only ever calibrated against this narrow synthetic distribution.

6. Generalizing the task: how many modes, particles, and features can it handle?

Generalized `train_four_mode_particle_filter.py` from a fixed 4-mode task to arbitrary `num_modes` (must be a power of 2 — the mode encoding is `num_regions` independent bits, `num_modes = 2^num_regions`; **M=3 is not representable** without a different, non-bit-based encoding, so M=3 was skipped rather than redesigned). Added a separate `num_modes=1` "trivial" floor case: query points reuse the *same* sign-threshold rule as the prior context, so the answer has zero hidden state — a genuine zero-ambiguity sanity check, structurally distinct from the region-based family (documented as "Option A" vs. the more unified but more disruptive "Option B" in `tfmplayground/experiments/N_MODE_TASK_FUTURE_PLANS.md`).

Trained one model across a curriculum of M values (per explicit feedback: *the same model should learn every setting, not a separate model per setting* — this happened twice, once for modes and once for features, and both times the first attempt was corrected mid-session):

- `mode_curriculum=(2,4,8)` , K=8, 1000 steps: **M=2 passes fully. M=4 near-passes (one mild violation). M=8 catastrophically collapses** (`distinct_slots_fraction=0.0` — the 8 particles never differentiate into 8 distinct hypotheses at all; `minimum_slot_joint_js=0.0` — literal duplicates). This is a capacity **cliff**, not a gradual degradation, and it sits between M=4 and M=8 for K=8 particles.

Extended to feature count (added `num_features` : feature 0 keeps the original signal, features 1..F-1 are pure-noise distractors):

- First attempt trained two *separate* models (F=1 vs F=8) — corrected per the same feedback as above; killed and redone as one curriculum.
- `mode_curriculum=(1,2,4)` × `feature_curriculum=(1,4,8)` , K=8: feature count degrades **gracefully** (M=2's incoherent-mass metric creeps 0.075→0.107 as F goes 1→8 — mild, not a cliff). Mode count remains the dominant limit: M=4 fails at every F, even F=1. **New, unexpected finding:** M=1 (the zero-ambiguity floor) is badly broken in this curriculum — accuracy stays perfect (0.996) but the gate reports ~99% ambiguity on a task with *provably zero* hidden state, at every feature count. Training on a mix of M values appears to teach the gate "there's always something to resolve," breaking its ability to recognize the genuinely unambiguous case.
- Added F=2 to the curriculum (`feature_curriculum=(1,2,4,8)`): M=4 unexpectedly **passed** at F=1,2,4 this time (only narrowly failed at F=8) — a large change from the previous run. Flagged as **likely training-run variance** (same total step budget, just a different schedule/RNG path), not a genuine causal effect of the value 2 — not yet confirmed by replication. The M=1 miscalibration was identical in both runs regardless of which feature values were included, which is the one part of this result I'd currently trust as real rather than noise.

7. Checking TabArena's actual feature-count range, and a second memory crash

Queried all 51 TabArena tasks directly: max feature count is 1,777 (`Bioresponse`), but the eval script's `max_n_features=500` cutoff excludes all of the >500-feature tasks, so the realistic ceiling among datasets this pipeline actually evaluates is **~95–112**. Chose a training ceiling of 100.

Bug found and fixed #3 (a second, smaller version of the earlier memory bug): training with a fixed discrete `feature_curriculum=(1,10,50,100)` pushed memory to **14–15GB and climbing** — `train_four_mode_particle_filter.py` had no `release_device_memory()` calls at all (unlike the eval script, which already had the fix from §1), so cycling through different feature-count tensor shapes kept stacking unreleased Metal buffers. Killed the run.

Fix: per a suggestion to sample feature count log-uniformly instead of from a small fixed list, replaced the discrete training-time `feature_curriculum` with continuous log-uniform sampling (`feature_log_range=(1,100)`) each episode, and added `release_device_memory()` calls after every backward pass. Verified: memory now stable at ~3GB instead of climbing past 14GB. A discrete `feature_curriculum=(1,10,50,100)` is kept *only* for evaluation reporting, so results are still comparable across specific feature counts even though training samples continuously.

A training run with `mode_curriculum=(2,4)` (M=3 skipped, per §6) and `feature_log_range=(1,100)` is in progress as of this writing; results not yet available.

Results so far

TabArena all-samples eval (§1), 15 real datasets, vanilla vs. this repo's models

- **vanilla nanoTabPFN**: best or tied-best on 12/15 datasets.
- **controlled** (2-hypothesis latent filter): wins on 3/15 (`hazelnut-spread-contaminant-detection` , `polish_companies_bankruptcy` , `seismic-bumps`), loses on the rest.
- **tabicl** (same architecture, trained on TabICL-prior episodes instead of the hand-designed "controlled" curriculum): loses on all 15, often by a wide margin.
- **adaptive_k4**: wins on 2/15 by a razor-thin margin (same two datasets `controlled` also won), otherwise indistinguishable from vanilla.

K=8 / K=16 (four-mode-trained checkpoints) on the same 15 datasets

Both win 6/15, both far more decisive than K=4 (mean α 0.65–0.76 vs. ~0.00006), both *worse* than vanilla on mean ROC AUC (confidence isn't calibrated — it's high on losses and wins alike).

512:512-retrained K=4 gate on the same 15 datasets

5/15 wins, mean $\alpha \approx 0.117$ (moderate), mean ROC AUC delta ≈ -0.0006 (near-neutral — best-calibrated of the checkpoints tested, not the biggest winner).

Synthetic curriculum diagnostics (four-mode-family task, not real data)

- K=8 trained on `mode_curriculum=(2,4,8)` : hard capacity cliff between M=4 and M=8 (M=8 = total particle-diversity collapse).
- K=8 trained on `mode_curriculum=(1,2,4)` × `feature_curriculum=(1,4,8)` : feature count degrades gracefully; mode count is the dominant constraint; the M=1 zero-ambiguity floor is robustly miscalibrated (~99% spurious ambiguity) regardless of feature count.
- Adding F=2 to the feature curriculum coincided with M=4 suddenly passing at most feature counts — likely training variance, not yet confirmed as a real effect.

Conclusions so far

1. **The original 256-row-context TabArena eval was measuring the wrong thing** — fixed, and the fix surfaced a genuine, reproducible memory bug (twice) in how this codebase's models handle large contexts under PyTorch's MPS backend, now fixed with `release_device_memory()`.
2. **None of the tested adaptive/particle models beat vanilla nanoTabPFN on average** on real TabArena data at large context. `controlled` and `adaptive_k4` come closest (small, dataset-specific wins); `tabicl` underperforms broadly; K=8/K=16 are more decisive but not better calibrated.
3. **The ambiguity gate's confidence and its correctness are not the same thing.** Every checkpoint tested is either uniformly timid (K=4/128:128) or uniformly bold (K=8/K=16) regardless of whether trusting the particle model would actually help on a given dataset — genuine per-dataset calibration hasn't been demonstrated by any variant so far, though the 512:512 retrain is the closest to "appropriately uncertain."
4. **Mode count (task complexity), not feature count, is the real capacity wall** for the K=8 particle architecture — a hard cliff around M=4–8, while added distractor features degrade performance only mildly and gradually.
5. **Curricula that mix task settings can introduce new failure modes that didn't exist when training on one setting at a time** — mixing multiple mode counts broke the model's ability to recognize the genuinely zero-ambiguity case, a regression invisible in any single-M evaluation.
6. **This entire synthetic-task family is 1-feature-native and only loosely connected to real tabular data** — the frozen backbone handles many features architecturally, but every gate/particle checkpoint tested so far was calibrated on a narrow synthetic distribution, which plausibly explains why none of them transfer their confidence calibration reliably to real data. The log-uniform feature-count training run (§7) is a direct attempt at closing part of that gap; results pending.